

SWIG MLIB Overview

Hans Roggeman



MHI

hans.roggeman@uk.mizuho-sc.com

10th March 2016

Simplified Wrapper and Interface Generator (SWIG)

- Open-source used to connect executables or libraries written in C or C++ with other languages
- 20+ languages supported across Unix, Windows and Mac
- <https://en.wikipedia.org/wiki/SWIG>
- Invented in 1996 <http://www.dabeaz.com/cv.html>

How does it connect the languages?

- using C++ code and an input file (*.i) SWIG generates language bindings for each language specifically
- the input file has language specific handling of the marshalling of data types, the cleanup of memory allocated by C++, the referral of SWIG templates to aid in creating the language bindings, etc.

exposed_functions.h & exposed_functions.cpp in swig_mlib project

Add the declaration in the header and implementation in the cpp. Do not use namespaces or non standard types on the function signature. Forward the implementation to the validation_api layer.

validation_api project in irsv_dev solution

Find an appropriate file that adequately categorizes the function you are about to add, if none exists, create the folder and/or file. keep the same function signature as in swig_mlib except add 'try' as a prefix to the function. Implement the function with validation of arguments.

Build

Build swig_mlib a DLL will be created as well as exported files (created in a pre-build step). In the case of java the necessary projects will be exported to the client directories (in post-build).

more detailed documentation

More detailed documentation can be found in the add_functions.txt and mlib_ReadMe.txt (for project settings)

```
exposed_functions.cpp X
exposed_functions.cpp H:\Workspace\MLIB\api\swig_mlib\swig_mlib\exposed_functions.cpp
(Global Scope) getDateFromTerm(const std::string & d

    }
    return "ILLEGAL STATE";
}

const std::string getDate(const std::string& dateString,
    const std::string& term,
    const std::string& slidingRule,
    const std::string& calendar) {
    try {
        return mlib::validation_api::tryGetDate(dateString,term,slidingRule,calendar);
        // this is the implementation (without validation and exception handling) that was forwarded to validation_api
        /*
        MBDDate mbDate(dateString.c_str(),"YYYYMMDD");
        MBString mbSlidingRule(slidingRule.c_str());
        MBString mbTerm(term.c_str());
        MBString mbCalendar(calendar.c_str());
        MBDDate mbReturnDate = MFirDateFuncUtility::getDate(mbDate,mbTerm,mbSlidingRule, mbCalendar);
        return std::string(mbReturnDate.stringWithFormat().getCString());
        */
    } catch (MError& mesx) {
        return mesx.getMsg();
    } catch (exception& ex) {
        return ex.what();
    }
    return "ILLEGAL STATE";
}
```

```
exposed_functions.cpp X
exposed_functions.cpp H:\Workspace\MLIB\api\swig_mlib\swig_mlib\exposed_functions.cpp
(Global Scope) getDateFromTerm(const std::string & d

    }
    return "ILLEGAL STATE";
}

const std::string getDate(const std::string& dateString,
    const std::string& term,
    const std::string& slidingRule,
    const std::string& calendar) {
    try {
        return mlib::validation_api::tryGetDate(dateString,term,slidingRule,calendar);
        // this is the implementation (without validation and exception handling) that was forwarded to validation_api
        /*
        MBDDate mbDate(dateString.c_str(),"YYYYMMDD");
        MBString mbSlidingRule(slidingRule.c_str());
        MBString mbTerm(term.c_str());
        MBString mbCalendar(calendar.c_str());
        MBDDate mbReturnDate = MFirDateFuncUtility::getDate(mbDate,mbTerm,mbSlidingRule, mbCalendar);
        return std::string(mbReturnDate.stringWithFormat().getCString());
        */
    } catch (MError& mesx) {
        return mesx.getMsg();
    } catch (exception& ex) {
        return ex.what();
    }
    return "ILLEGAL STATE";
}
```

```
tryDateFunctions.cpp x
tryDateFunctions.cpp H:\Workspace\MLIB\mllib\src\validation_api\src\tryDateFunctions.cpp
{} mlib

    }
}

const std::string tryGetDate(const std::string& dateString,
    const std::string& term,
    const std::string& slidingRule,
    const std::string& calendar) {

    // Validation still needs to be implemented here
    MBDate mbDate(dateString.c_str(), "YYYYMMDD");
    MBString mbSlidingRule(slidingRule.c_str());
    MBString mbTerm(term.c_str());
    MBString mbCalendar(calendar.c_str());
    MBDate mbReturnDate = MFirDateFuncUtility::getDate(mbDate, mbTerm, mbSlidingRule, mbCalendar);
    return std::string(mbReturnDate.stringWithFormat().getCString());
}

}

}
```